المكتبة البرمجية وإطار العمل

من يستدعي من؟

بحث في الفرق بين Library و Framework مع أمثلة تطبيقية ورسوم بيانية

زياد أنور المقطري

تمهيد

حين كُلّفت بهذا البحث ظننت أن الأمر لن يتجاوز صفحة أو صفحتين، فالفرق بين المكتبة والإطار يبدو من الأشياء التي «يعرفها الجميع». لكن بمجرد أن بدأت أبحث في Google وأقرأ ما كُتب عن الموضوع، اكتشفت أن المسألة أعمق مما توقعت، وأن كثيرًا من الشروح المنتشرة تتعارض فيما بينها، بل إن بعضها يستخدم التشبيه نفسه ليصل إلى نتيجة معاكسة.

يحاول هذا البحث أن يجيب عن سؤال واحد بشكل واضح: **ما الذي يجعل أداة برمجية ما مكتبةً، وأخرى إطارَ عمل؟** وسأحاول أن أبتعد قدر الإمكان عن التعريفات المحفوظة، وأن أقترب من الأمثلة العملية والأرقام، لأن الفكرة تتضح بالكود أكثر مما تتضح بالكلام.

اعتمدتُ في جمع المادة على البحث في Google عن المصطلحات الإنجليزية والعربية، ثم ركزت على المصادر الأصلية حين وجدتها، مثل مقال Martin Fowler عن انقلاب التحكم، وتوثيق Spring، والموقع الرسمي لـ React، إلى جانب مقالات تعليمية من freeCodeCamp وGeeksforGeeks وInterviewBit. أما الأرقام فمصدرها استبيان Stack Overflow للمطورين 2025 وموقع W3Techs المتخصص في إحصاءات تقنيات الويب.

محتويات البحث: أولًا: لماذا نخلط بين المصطلحين؟ · ثانيًا: من يستدعي من؟ · ثالثًا: مثال من عالم Python · رابعًا: نظرة تاريخية · خامسًا: طيف لا خطان · سادسًا: React بين التصنيفين · سابعًا: مقارنة على سبعة معايير · ثامنًا: ماذا تقول الأرقام؟ · تاسعًا: درس AngularJS · عاشرًا: خطوات عملية للاختيار · الخلاصة · المصادر.

أولًا: لماذا نخلط بين المصطلحين؟

السبب الأول بسيط: كلاهما «كود كتبه شخص آخر لنستفيد منه». فالمكتبة والإطار يشتركان في الغاية نفسها، وهي توفير حلول جاهزة لمشكلات متكررة حتى لا يعيد كل مبرمج اختراع العجلة. ومن هذه الزاوية يبدو التفريق بينهما ترفًا نظريًا.

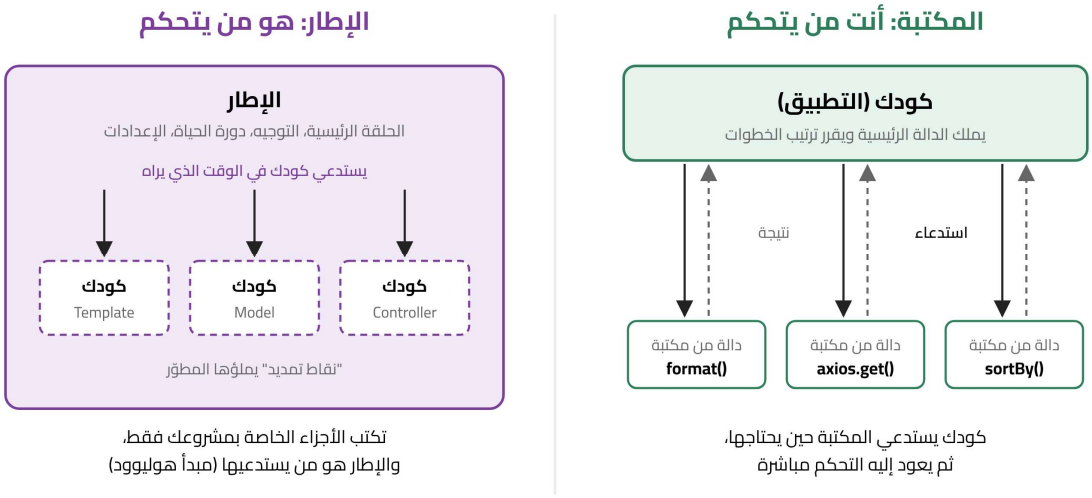
السبب الثاني أن الشروح المنتشرة تعتمد كثيرًا على التشبيهات، والتشبيهات قد تضلل. ومن الطريف أنني وجدت مصدرين مشهورين يستخدمان تشبيه «البيت» بطريقتين متعاكستين تقريبًا. مقال freeCodeCamp يشبّه المكتبة بالذهاب إلى متجر أثاث: بيتك موجود أصلًا، وأنت تختار منه قطعة هنا وقطعة هناك، فأنت المتحكم. ويشبّه الإطار ببناء منزل نموذجي وفق مخطط جاهز وخيارات محدودة، فالمقاول والمخطط هما المتحكمان. أما مقال InterviewBit فيشبّه المكتبة ببناء بيتك من الأساس بالطراز الذي تريده، والإطار بشراء بيت جديد جاهز.

التشبيهان يحاولان قول الشيء نفسه، لكن القارئ المبتدئ قد يخرج منهما بانطباعين مختلفين: هل المكتبة هي «البناء من الصفر» أم «الإضافة إلى الموجود»؟ ولهذا أفضّل أن نترك التشبيهات جانبًا، ونذهب مباشرة إلى المعيار التقني الذي تتفق عليه المصادر كلها.

والسبب الثالث أن الجهات الكبرى نفسها لا تلتزم بالفصل الدقيق. فاستبيان Stack Overflow مثلًا يجمع تحت سؤال واحد بعنوان «أطر وتقنيات الويب» أدوات من أنواع مختلفة تمامًا: Node.js وهو بيئة تشغيل، وReact وهو مكتبة، وWordPress وهو نظام لإدارة المحتوى، إلى جانب أطر حقيقية مثل Django وSpring Boot.

ثانيًا: من يستدعي من؟

الفكرة التي تحسم الموضوع تتلخص في سؤال: **من يستدعي من؟** عندما تستخدم مكتبة، فأنت من يستدعي دوالها في المكان والوقت الذي تختاره، ثم يعود إليك التحكم. وعندما تستخدم إطارًا، فأنت تكتب أجزاء من الكود وتضعها في الأماكن التي يحددها الإطار، ثم يتولى الإطار استدعاءها حين يرى ذلك مناسبًا. هذه الظاهرة اسمها **انقلاب التحكم** (Inversion of Control).



شكل (1): الفرق بين اتجاه الاستدعاء في المكتبة والإطار

يكتب Martin Fowler أن انقلاب التحكم ظاهرة شائعة تصادفها عند تمديد الأطر، وأنها كثيرًا ما تُعدّ السمة المميزة للإطار. ويصف المكتبة بأنها في الأساس مجموعة من الدوال يمكنك استدعاؤها، كل استدعاء يؤدي عملًا ثم يعيد التحكم للعميل. أما الإطار فيلعب في الغالب دور البرنامج الرئيسي الذي ينسق أنشطة التطبيق ويرتبها، وهذا ما يمنحه القدرة على أن يكون «هيكلًا عظميًا» قابلًا للتمديد.

مثال Fowler: سطر الأوامر مقابل النافذة

أوضح مثال وجدته على الفكرة هو المثال الذي يذكره Fowler نفسه، وسأعيد كتابته هنا بلغة Python. في النسخة الأولى، البرنامج يعمل في سطر الأوامر، والكود هو الذي يقرر متى يسأل المستخدم ومتى يقرأ الإجابة ومتى يعالجها:

```
name = input('What is your name? ')
process_name(name)
quest = input('What is your quest? ')
process_quest(quest)
our code decides the order of every step #
```

في النسخة الثانية نستخدم واجهة رسومية بمكتبة Tkinter. نحن نعرّف الحقول ونربط كل حقل بدالة، ثم نسلّم الأمر لـ mainloop. من تلك اللحظة لم نعد نعرف متى ستُستدعى process_name أو process_quest؛ نظام النوافذ هو من يقرر ذلك حسب ما يفعله المستخدم:

```
import tkinter as tk


()root = tk.Tk
name_box = tk.Entry(root)
name_box.bind('<FocusOut>', lambda e: process_name(name_box.get()))
quest_box = tk.Entry(root)
quest_box.bind('<FocusOut>', lambda e: process_quest(quest_box.get()))


root.mainloop()   # control is handed over; Tk calls us, not the other way
```

هذا هو «الانقلاب» بالضبط. في الحالة الأولى نحن نستدعي، وفي الثانية نحن نُستدعى. ويذكر Fowler مثالًا آخر من عالم الاختبارات، هو إطار JUnit، حيث يكتب المطور دالتي setUp وtearDown، والإطار هو من يستدعيهما قبل كل اختبار وبعده لتجهيز البيئة وتنظيفها. المطور يحدد **ماذا** تفعل الدالة، أما **متى** تُنفّذ فهذا قرار الإطار.

مبدأ هوليوود

لهذه الفكرة اسم آخر أكثر طرافة هو «مبدأ هوليوود»: «لا تتصل بنا، سنتصل بك». وهي العبارة التي تُقال للممثلين بعد تجارب الأداء. وتربطه بعض المراجع بمبدأ عكس الاعتماديات، وتوضحه بمثال صفحات ASP.NET Web Forms، حيث يكتب المطور دوال تستجيب لأحداث مثل تحميل الصفحة أو الضغط على زر، بدل أن يتحكم هو في تشغيل خادم الويب ويتخذ كل القرارات بنفسه.

ثالثًا: مثال من عالم Python

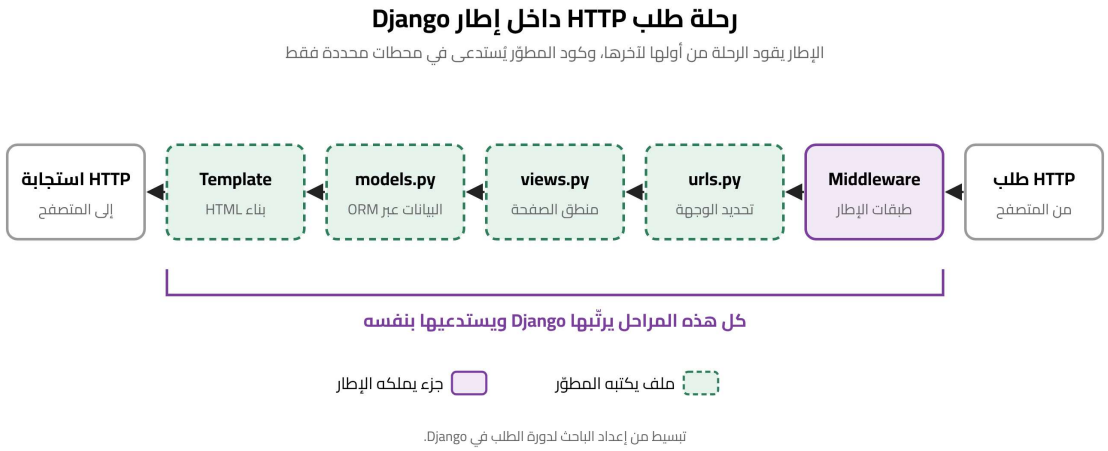
لنأخذ مثالًا يعرفه أغلب طلاب البرمجة. مكتبة NumPy مكتبة بامتياز: تستوردها، وتستدعي منها ما تحتاج من عمليات على المصفوفات، وتستمر في كتابة برنامجك كما تريد. لا تفرض عليك شكلًا للمشروع، ولا تهتم بمكان ملفاتك ولا بطريقة تسميتها.

```
import numpy as np


grades = np.array([88, 95, 72, 60])
print(grades.mean())   # we call NumPy whenever we like
```

أما Django فشيء مختلف تمامًا. عندما يصل طلب من المتصفح، فإن Django هو من يستقبله، ويمرره على طبقات الوسائط (Middleware)، ثم يبحث في ملف urls.py الذي كتبه المطور ليعرف أي دالة يجب أن تتعامل مع هذا الرابط، ثم يستدعي دالة العرض في views.py، والتي قد

تستخدم النماذج المعرّفة في models.py، ثم يبني صفحة HTML من القالب ويعيدها للمتصفح. المطور كتب هذه الملفات، لكنه لم يستدعِ أيًّا منها بنفسه.



شكل (2): رحلة الطلب في Django: الإطار يقود والكود الخاص بالمطور يُستدعى في محطات محددة

```
views.py  - Django decides when this runs #
from django.http import JsonResponse
from .models import Student


:def student_list(request)
data = list(Student.objects.values('name', 'grade'))
return JsonResponse(data, safe=False)
```

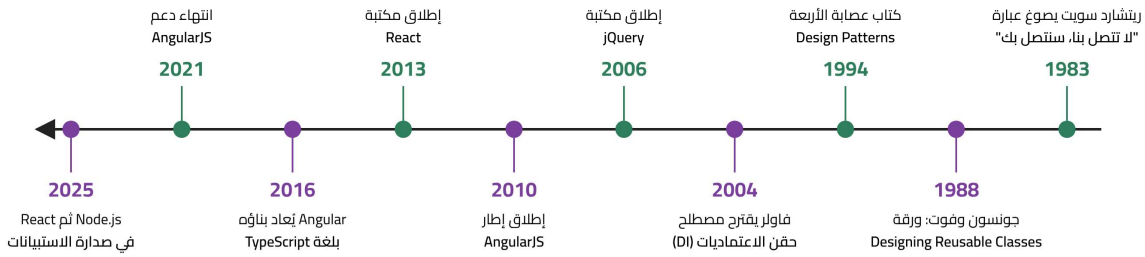
ويوصف Django عادة بأنه إطار «البطاريات مضمّنة» (Batteries included)، أي أنه يأتي ومعه تقريبًا كل ما يحتاجه المشروع: نظام ORM للتعامل مع قواعد البيانات، ولوحة إدارة جاهزة، ونظام مصادقة، ونظام توجيه وقوالب. وعلى النقيض منه يقف Flask، الذي يسمى «إطارًا مصغرًا» لأن نواته لا تضم إلا الأساسيات: توجيه الروابط ومعالجة الطلبات ومحرك القوالب. وكلاهما إطار بالمعنى الدقيق، لأن كليهما يملك دورة الطلب ويستدعي كود المطور، لكن Flask يترك للمطور حرية أكبر في اختيار بقية القطع.

وقد لفت انتباهي تعليق في أحد المقالات المقارنة يلخص الجدل بين الفريقين بشكل ظريف: أنصار Django يقولون إن مطور Flask سينتهي به الأمر إلى إعادة بناء Django بنفسه، ولكن بشكل أسوأ وأبطأ، وأنصار Flask يردون بأن Django ثقيل جدًا وأصعب في التخصيص عند التوسع. وفي الحقيقة كلا الرأيين فيه جانب من الصواب، وهذا بالضبط جوهر الموازنة بين المرونة والبنية الجاهزة.

رابعًا: نظرة تاريخية سريعة

قد يظن البعض أن هذا التمييز ظهر مع أطر الويب الحديثة، لكنه أقدم من ذلك بكثير. فبحسب Fowler، ظهر مصطلح «انقلاب التحكم» أول مرة في ورقة Ralph Johnson وBrian Foote بعنوان «Designing Reusable Classes» عام 1988. وقد عرّف الباحثان الإطار في تلك الورقة بأنه مجموعة من الأصناف تجسد تصميمًا مجردًا لحلول عائلة من المشكلات المتقاربة، وأنه يتيح إعادة استخدام أوسع من الصنف الواحد. ثم انتقل المصطلح إلى كتاب «عصابة الأربعة» عن أنماط التصميم عام 1994. أما عبارة «مبدأ هوليوود» فيعيدها Fowler إلى ورقة لـ Richard Sweet عن بيئة Mesa عام 1983.

خط زمني لأهم المحطات المرتبطة بالموضوع



المصادر: مقال Martin Fowler عن انقلاب التحكم (2005)، توثيق AngularJS، توثيق React، استبيان Stack Overflow 2025.

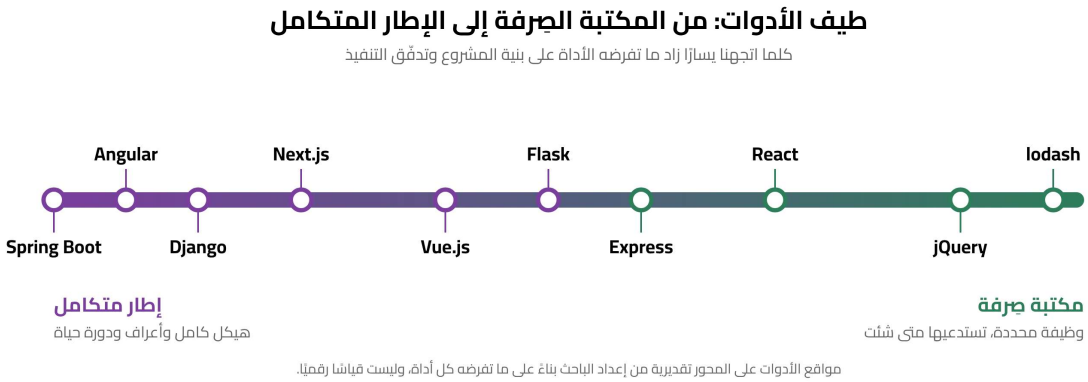
شكل (3): أبرز المحطات الزمنية المرتبطة بموضوع البحث

ومع انتشار ما يسمى «حاويات انقلاب التحكم» في عالم Java، صار المصطلح يُفهم بمعنى ضيق. ولهذا اقترح Fowler عام 2004 تسمية أدق لهذا النمط الخاص هي «حقن الاعتماديات» (Dependency Injection). ويؤكد توثيق Spring هذه العلاقة حين يصف حقن الاعتماديات بأنه شكل متخصص من انقلاب التحكم، حيث تُعرّف الكائنات ما تحتاجه من كائنات أخرى، ثم تتولى الحاوية حقنها عند الإنشاء. فالكائن لم يعد يبحث عن اعتمادياته أو ينشئها بنفسه، بل تأتيه جاهزة.

خامسًا: طيف لا خطان

بعد قراءة عدد كبير من المقالات وصلت إلى قناعة أن تقسيم الأدوات إلى «مكتبات» و«أطر» بشكل حاد فيه شيء من التبسيط. الأقرب للواقع أن نتخيل طيفًا متصلًا: في طرفه الأيمن مكتبات

صرفة مثل lodash لا تفرض عليك شيئًا، وفي طرفه الأيسر أطر متكاملة مثل Spring Boot وAngular تفرض بنية المشروع وأسلوب كتابته. وما بينهما أدوات تفرض قدرًا متفاوتًا من البنية.



شكل (4): طيف الأدوات بين المكتبة والإطار (المواقع تقديرية)

ويرتبط بهذا الطيف وصف شائع هو أن الإطار «صاحب رأي» (Opinionated)، أي أنه يملي عليك طريقة معينة لفعل الأشياء. ويرى كاتب مقال freeCodeCamp أن الأطر تميل بطبيعتها لهذا، لأن قبولك بانقلاب التحكم يعني أنك تتنازل عن جزء من حريتك في التصميم. ويضيف أن الحكم على درجة «الرأي» مسألة ذاتية، فهو شخصيًا يرى Angular شديد الإلزام، وVue.js أخف منه بكثير.

سادسًا: React بين التصنيفين

لا يكتمل أي بحث في هذا الموضوع دون التوقف عند React، لأنه السؤال الذي يتكرر في مقابلات العمل تقريبًا: هل React مكتبة أم إطار؟ الإجابة الرسمية واضحة؛ فالموقع الرسمي يقدّم React على أنه مكتبة لواجهات المستخدم على الويب والتطبيقات الأصلية، ويقول صراحة في صفحته الرئيسية إن React مكتبة، ثم يوصي من يريد بناء تطبيق كامل باستخدام إطار مبني عليه مثل Next.js.

لكن لماذا يستمر الجدل؟ لأن React في الواقع يمارس قدرًا من انقلاب التحكم: أنت تكتب المكوّن، وReact هو من يقرر متى يرسمه ومتى يعيد رسمه. ومن جهة أخرى، يبقى نطاق هذا التحكم محصورًا في طبقة العرض، أما التوجيه وجلب البيانات وإدارة الحالة وتنظيم المشروع فتُترك

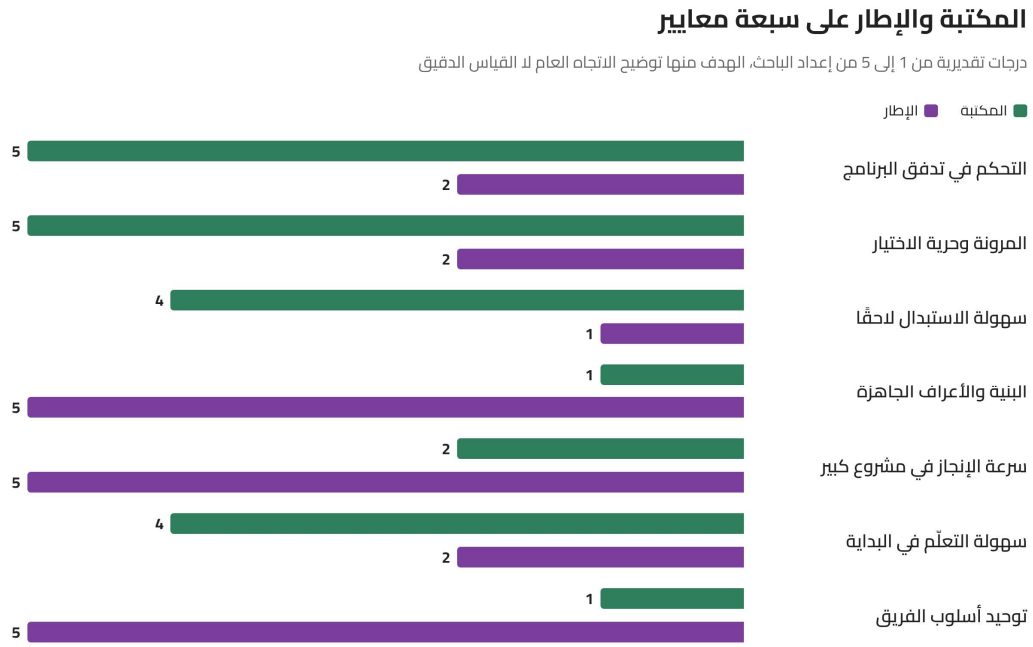
لاختيار المطور. ولهذا يجد مطور React نفسه أمام قرارات كثيرة: أي مكتبة يستخدم للنماذج؟ أي مكتبة للتوجيه؟ أي أداة للاختبار؟ وهذا بالضبط ما لا يحدث مع إطار متكامل مثل Angular.

رأيي الشخصي بعد هذه القراءة أن تسمية React «مكتبة» هي الأدق، مع الاعتراف بأن المنظومة المحيطة به (React مع Next.js أو مع React Router ومكتبات إدارة الحالة) تتصرف في المشاريع الحقيقية كإطار متكامل. أي أن الخلاف في الغالب ليس على React نفسه، بل على ما يُقصد بكلمة React في السياق.

سابعًا: مقارنة على سبعة معايير

حاولتُ أن أجمع الفروق العملية في جدول واحد، ثم حوّلتها إلى رسم بياني بدرجات تقديرية. أنبّه إلى أن الدرجات ليست نتيجة قياس، بل هي طريقتي في تلخيص ما قرأته:

المعيار	المكتبة	الإطار
التحكم في التدفق	بيد المطور بالكامل	بيد الإطار، والمطور يملأ الفراغات
المرونة	عالية؛ تختار ما تشاء وتتركه متى شئت	محدودة بالأعراف التي يفرضها الإطار
الاستبدال لاحقًا	ممكن بجهد معقول	مكلف جدًا وقد يعني إعادة الكتابة
البنية الجاهزة	شبه معدومة	كاملة: مجلدات وتسميات ودورة حياة
سرعة الإنجاز في مشروع كبير	أبطأ في البداية لكثرة القرارات	أسرع لأن كثيرًا من القرارات محسومة
التعلم في البداية	أسهل لأن النطاق صغير	أصعب لأنك تتعلم طريقة تفكير كاملة
توحيد أسلوب الفريق	يحتاج اتفاقًا داخليًا واضحًا	يفرضه الإطار تلقائيًا

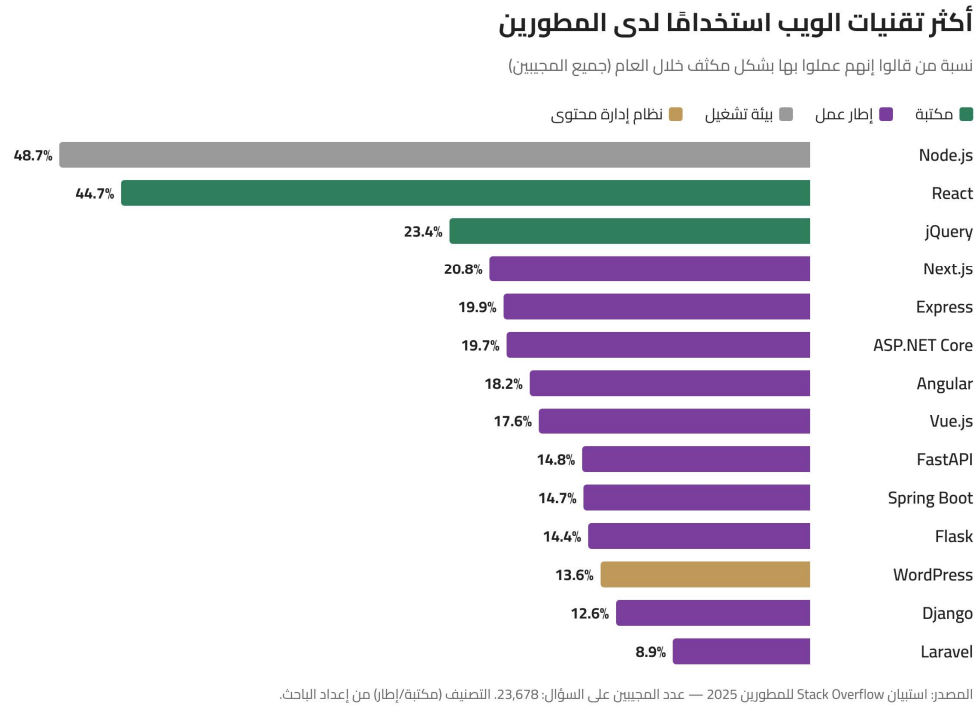


شكل (5): درجات تقديرية للمكتبة والإطار على سبعة معايير

الملاحظ في الرسم أن كل نقطة قوة لطرف تقابلها نقطة ضعف عند الطرف الآخر تقريبًا. وهذا ما جعلني أقتنع بأن السؤال «أيهما أفضل؟» سؤال غير دقيق، والأصح أن نسأل: «أيهما أنسب لهذا المشروع، وهذا الفريق، وهذه المدة؟».

ثامنًا: ماذا تقول الأرقام؟

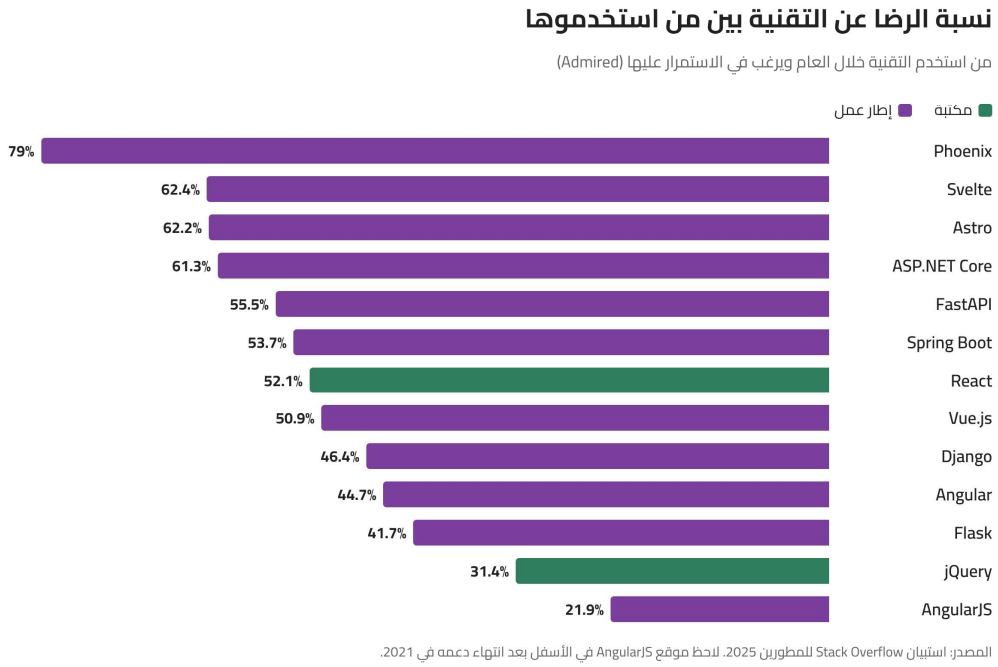
شارك في استبيان Stack Overflow للمطورين لعام 2025 أكثر من 49 ألف مطور، وأجاب 23,678 منهم عن سؤال تقنيات وأطر الويب التي عملوا عليها بشكل مكثف خلال العام. وقد أعدتُ تصنيف النتائج إلى مكتبات وأطر وغيرها، فكانت الصورة كما في الشكل التالي:



شكل (6): نسبة استخدام تقنيات الويب حسب استبيان Stack Overflow 2025، مع تصنيف الباحث

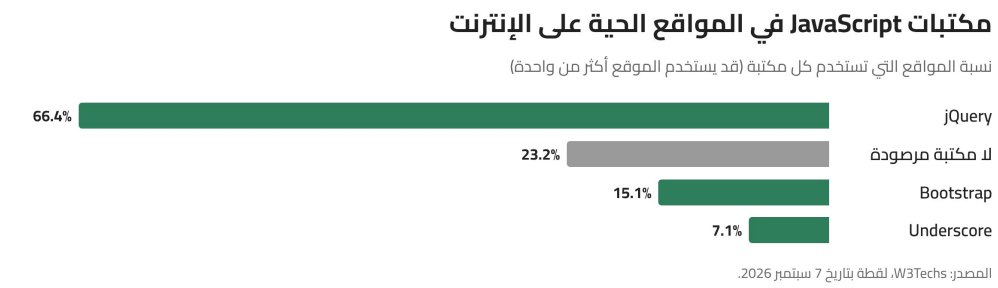
ما استوقفني هنا أمران. الأول أن مكتبتين فقط (React بنسبة 44.7% وjQuery بنسبة 23.4%) تتفوقان على كل الأطر المدرجة دون استثناء. والثاني أن الأطر نسبها متقاربة جدًا (بين 8.9% و20.8%)، وهذا مفهوم لأن كل إطار مرتبط بلغة معينة: Next.js وExpress وAngular وVue.js لعالم JavaScript، وSpring Boot لعالم Java، وASP.NET Core لعالم .NET.، وFastAPI وFlask وDjango لعالم Python. فالمكتبة تنتشر أفقيًا عبر المشاريع، بينما يتوزع جمهور الأطر على بيئات مختلفة.

ومن الأرقام اللافتة في الاستبيان أن FastAPI سجّل زيادة بخمس نقاط مئوية، وهي من أبرز التحولات في هذا القسم، ويفسّر الاستبيان ذلك بقوة منظومة Python واتجاه المطورين لاستخدامها في بناء الواجهات البرمجية.



شكل (7): نسبة الرضا (Admired) بين من استخدموا التقنية حسب استبيان Stack Overflow 2025

أما مؤشر الرضا فيحكي قصة أخرى. Phoenix، وهو إطار للغة Elixir، يتصدر بنسبة 79% مع أن استخدامه قليل، وهو الأعلى رضًا منذ 2023. وفي أسفل القائمة يأتي AngularJS بنسبة 21.9% فقط، ثم jQuery بنسبة 31.4%. وهذان الرقمان يكشفان شيئًا مهمًا: أن الأداة قد تبقى مستخدمة على نطاق واسع لا لأن الناس يحبونها، بل لأن الخروج منها مكلف.



شكل (8): مكتبات JavaScript في المواقع الحية حسب W3Techs، سبتمبر 2026

وتؤكد إحصاءات W3Techs هذه الفكرة من زاوية مختلفة. ففي لقطة بتاريخ 7 سبتمبر 2026، كانت jQuery مستخدمة في 66.4% من جميع المواقع، وهو ما يعادل 86.4% من المواقع التي تستخدم مكتبة JavaScript يرصدها الموقع، بينما لا تستخدم 23.2% من المواقع أيًّا من

المكتبات المرصودة. وهذا الانتشار الهائل لمكتبة قديمة نسبيًا يرجع في رأيي إلى طبيعتها كمكتبة: يمكن إضافتها لأي صفحة بسطر واحد، ولا تتطلب إعادة بناء الموقع، ولا تفرض عليه شكلًا معينًا.

تاسعًا: درس AngularJS

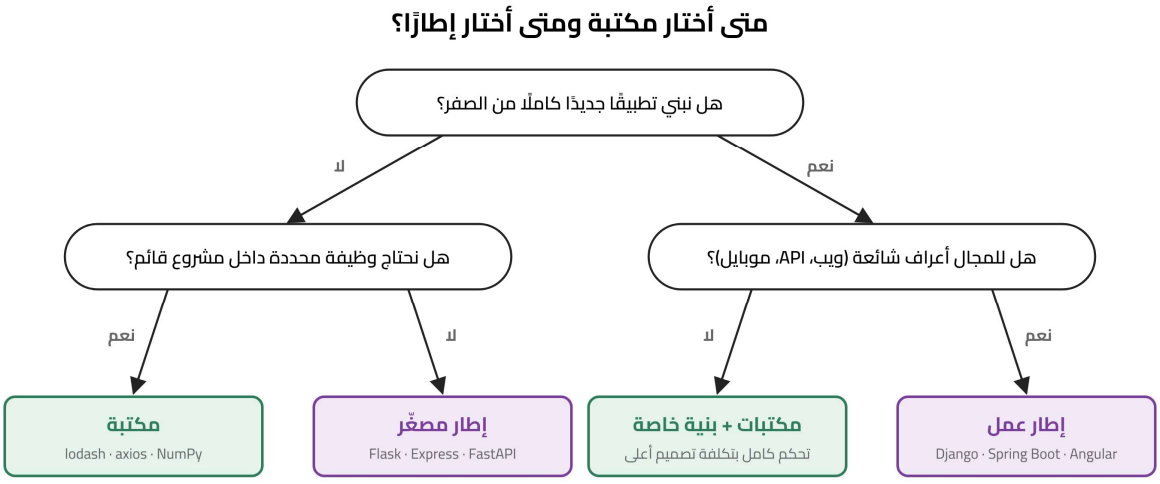
إذا أردنا مثالًا واقعيًا على الوجه الآخر للأطر، فلن نجد أوضح من AngularJS. هذا الإطار كان في وقته من أشهر أطر الواجهات الأمامية، ثم انتهى دعمه رسميًا في 31 ديسمبر 2021، ولم تصدر له منذ ذلك التاريخ أي تحديثات أمنية أو إصلاحات من Google.

قد يقول قائل: لماذا لا تنتقل الشركات ببساطة إلى Angular الحديث؟ والجواب أن Angular الحديث أُعيدت كتابته بلغة TypeScript عام 2016 على أسس مختلفة، ولذلك فإن «الانتقال» إليه هو في الحقيقة إعادة بناء للتطبيق. وتذكر شركة OpenLogic في مقال لها عام 2024 أن أكثر من 20% من المؤسسات الكبيرة لم تكن قد انتقلت إلى إطار آخر حتى ذلك الوقت، كما تصف Angular نفسه بأنه يحمل قدرًا كبيرًا من الارتباط بالمورّد (Vendor lock-in) وأن كتابة أجزاء مستقلة عن الإطار فيه صعبة جدًا.

الخلاصة من هذه الحالة أن الإطار حين يتولى التحكم، فإنه يتغلغل في كل ملف من ملفات المشروع. ولو كان الكود مبنيًا على مكتبات منفصلة، لكان من الممكن استبدال كل مكتبة على حدة. هذا لا يعني تجنب الأطر، بل يعني أن نختارها بعين على المستقبل، وأن نفصل منطق العمل الأساسي عن الإطار قدر المستطاع.

عاشرًا: خطوات عملية للاختيار

من خلال ما سبق، حاولتُ أن أضع مجموعة أسئلة بسيطة تساعد على الاختيار، ورسمتها في المخطط التالي:



شكل (9): مخطط مقترح لاختيار المكتبة أو الإطار

1. حدّد أولًا: هل تبني تطبيقًا كاملًا جديدًا، أم تضيف وظيفة إلى مشروع قائم؟ في الحالة الثانية تكون المكتبة غالبًا هي الحل.
2. إن كان التطبيق جديدًا، فاسأل: هل لهذا النوع من التطبيقات أعراف مستقرة، كتطبيقات الويب أو الواجهات البرمجية؟ إن كان كذلك فالإطار يوفر عليك قرارات كثيرة.
3. انظر إلى الفريق: الفريق الكبير يستفيد من الإطار لأنه يوحد أسلوب الكتابة، والمطور الفرد قد يفضل حرية المكتبات أو الأطر المصغرة.
4. فكّر في عمر المشروع: المشروع الذي سيعيش سنوات يحتاج إطارًا ذا مجتمع نشط وخطة دعم واضحة، حتى لا يتكرر سيناريو AngularJS.
5. لا تتجاهل الحل الوسط: الأطر المصغرة مثل Flask وExpress وFastAPI تمنحك هيكلًا خفيفًا مع حرية كبيرة في اختيار بقية الأدوات.

الخلاصة

بدأت هذا البحث وأنا أظن أن الفرق بين المكتبة والإطار مسألة حجم، وانتهيت إلى أن الحجم لا علاقة له تقريبًا بالموضوع. المعيار الحقيقي هو اتجاه التحكم: **أنت تستدعي المكتبة، والإطار**

يستدعيك. وهذه الفكرة ليست اختراعًا حديثًا، بل تعود إلى ثمانينيات القرن الماضي، وتطورت لاحقًا إلى أنماط مثل حقن الاعتماديات التي تقوم عليها أطر كبرى مثل Spring.

كما تبين أن الأدوات الحقيقية لا تنقسم دائمًا إلى فئتين واضحتين، بل تتوزع على طيف، وأن الخلاف حول أدوات مثل React يرجع في جزء كبير منه إلى اختلاف ما نقصده بالاسم: المكتبة نفسها أم المنظومة المبنية حولها. وأظهرت الأرقام أن المكتبات تنتشر انتشارًا واسعًا بسبب سهولة إضافتها، وأن الأطر تتوزع بحسب اللغات والبيئات، وأن الارتباط بإطار ما قد يتحول إلى عبء حين ينتهي دعمه.

وإن كان لي أن أخرج بنصيحة واحدة من هذا البحث فهي: قبل أن تسأل «هل هذه الأداة جيدة؟»، اسأل «من سيتحكم في مشروعي إن اخترتها، وهل أنا مستعد لذلك؟».

المصادر

[1] .Fowler, M. (2005). Inversion Of Control https://martinfowler.com/bliki/InversionOfControl.html

[2] Johnson, R. E., & Foote, B. (1988). Designing Reusable Classes. Journal of .Object-Oriented Programming, 1(2), 22–35

[3] :Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns .Elements of Reusable Object-Oriented Software. Addison-Wesley

[4] .Spring. Introduction to the Spring IoC Container and Beans https://docs.spring.io/spring/reference/core/beans/introduction.html

[5] React. React – The library for web and native user interfaces. https://react.dev

[6] .Stack Overflow. (2025). 2025 Developer Survey – Technology https://survey.stackoverflow.co/2025/technology

[7] .W3Techs. (2026). Usage statistics and market shares of JavaScript libraries https://w3techs.com/technologies/overview/javascript_library

[8] .freeCodeCamp. The difference between a framework and a library https://www.freecodecamp.org/news/the-difference-between-a-framework-and-/a-library-bd133054023f

[9] ?freeCodeCamp. Is React a library or a framework /https://www.freecodecamp.org/news/is-react-a-library-or-a-framework

[10] .InterviewBit. Framework vs Library: Full Comparison /https://www.interviewbit.com/blog/framework-vs-library

[11] .GeeksforGeeks. (2023). Software Framework vs Library /https://www.geeksforgeeks.org/software-framework-vs-library

[12] DevIQ. Hollywood Principle. https://deviq.com/principles/hollywood-principle

[13] Kenneth Reitz. Flasky Goodness (talk). https://kennethreitz.org/talks/flasky-goodness.pdf

[14] OpenLogic by Perforce. (2024). How to Choose a JavaScript Framework to Replace AngularJS. https://www.openlogic.com/blog/how-to-choose-javascript-framework-replace-angularjs

[15] endoflife.ai. AngularJS Is Long Dead: Migration Options That Actually Work in https://endoflife.ai/article-migrate-angularjs .2026